RELATÓRIO TÉCNICO / ROBOTIC PROCESS AUTOMATION

Passo Firme

Triagem automática de relatos de barreiras em calçadas

ESCOPO IMPLEMENTADO	Integração HTTP real com API fictícia local, processamento em lote, regras de decisão, auditoria e testes automatizados.
AINDA PENDENTE	Conectar ao MVP público do grupo, preencher integrantes, publicar no GitHub e gravar o vídeo de até 3 minutos.

Equipe: [NOME COMPLETO / RM] — preencher todas as pessoas do grupo.

Disciplina: Robotic Process Automation (RPA)
Projeto CitizenTech: Passo Firme
Entrega da disciplina: 19/11/2026 às 23h55, via Teams
Data de elaboração: novembro de 2026

Natureza da evidência: protótipo executável e testes locais com dados totalmente sintéticos. Este documento não apresenta a API simulada como se fosse a aplicação em produção.

01 Identificação da CitizenTech e problema

O **Passo Firme** é uma proposta CitizenTech para facilitar o registro, a visualização e a priorização de barreiras em calçadas de São Paulo. O público beneficiado inclui idosos, pessoas com deficiência e pedestres que dependem de deslocamentos a pé. O projeto prevê aplicativo web acessível, API, detecção visual, assistente de orientação, RPA, processamento analítico e robô vistoriador.

Problema operacional: no processo manual, um operador precisa ler relatos individualmente, verificar foto e localização, identificar duplicidades, estimar risco, consultar o encaminhamento cabível e atualizar o status. Essa rotina é repetitiva, sujeita a atrasos e inconsistências.

02 Objetivo da automação e benefício ao cidadão

Automatizar a triagem de relatos recebidos, com critérios transparentes de validade, duplicação, confiança e prioridade, de forma que barreiras potencialmente graves sejam encaminhadas mais rapidamente à fila responsável. Casos incertos são enviados para revisão humana. O benefício esperado é reduzir retrabalho e padronizar a priorização, sem substituir a decisão final das autoridades.

03 Processo manual e processo automatizado

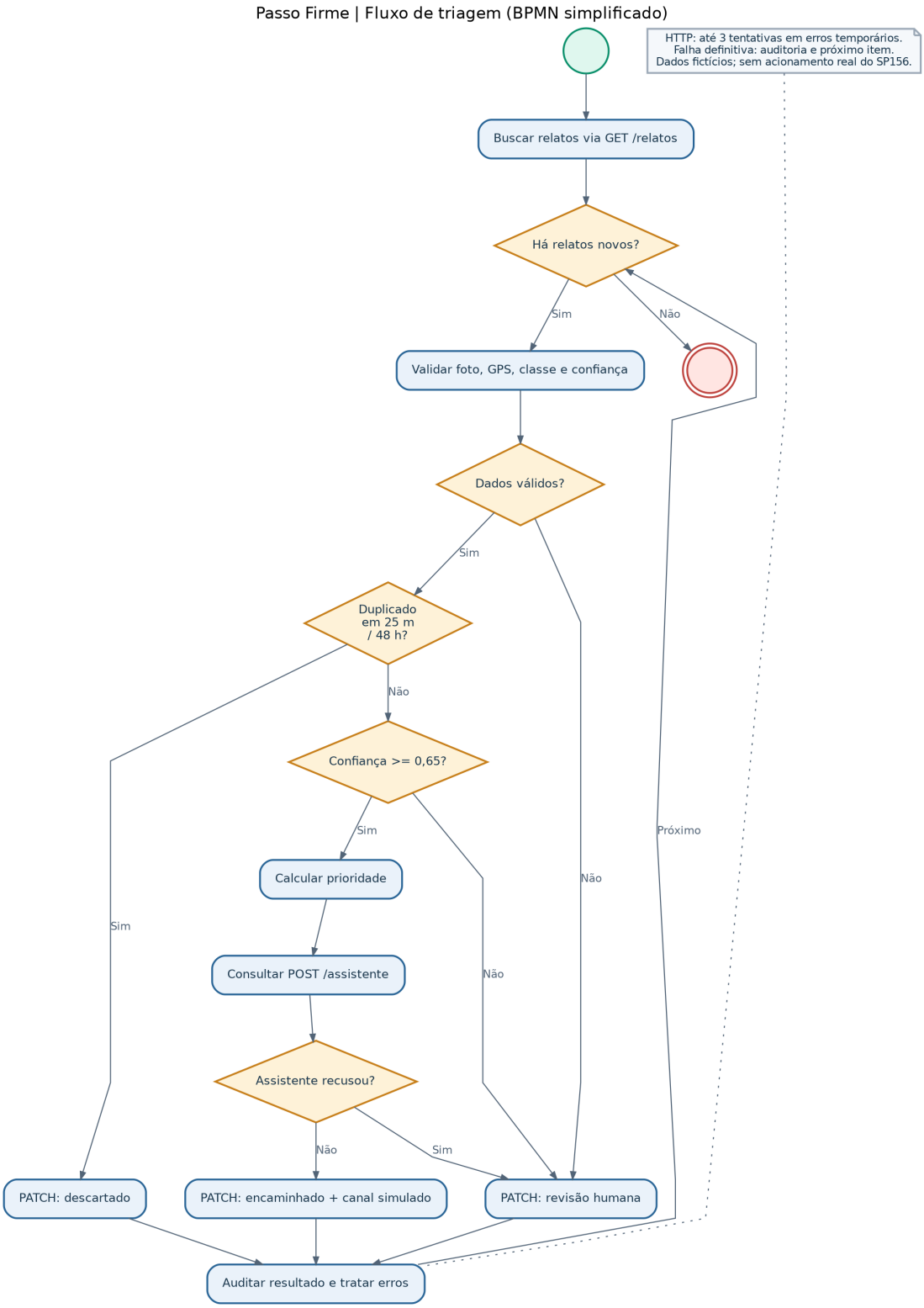
Processo manual (AS-IS)	Processo automatizado (TO-BE)
Ler cada relato e conferir campos.	GET /relatos busca o lote; validação automática de URL, GPS, classe e confiança.
Comparar relatos para encontrar duplicados.	Heurística: mesma classe, até 25 m de distância e até 48 h.
Definir urgência e consultar orientação.	Pontuação determinística e consulta ao POST /assistente.
Atualizar planilha/sistema manualmente.	PATCH /relatos/{id}/status e JSON estruturado para canal simulado.
Anotar resultados e falhas sem padrão.	Auditoria JSONL por item e por execução, inclusive falhas.

04 Tecnologias utilizadas e justificativa

Python 3.10+ (biblioteca padrão) foi escolhido por simplicidade de implantação e integração via HTTP. O robô é independente do framework da API e pode conversar com o FastAPI/PostgreSQL previstos pelo grupo. **JSON** permite transferir informações estruturadas para a API e o painel. **unittest** testa regras e integrações, enquanto o **Graphviz** gera a representação visual do processo. O arquivo BPMN 2.0 é incluído separadamente.

05 Fluxo em BPMN — etapas, decisões, repetição e exceções

O fluxo representa os eventos de início e fim, as atividades automáticas, os gateways de decisão e o retorno para o próximo item do lote. A representação é simplificada; o arquivo **docs/fluxo-rpa.bpmn** contém os elementos editáveis do processo.



Fonte: diagrama elaborado para o protótipo do Passo Firme. O envio ao SP156 permanece simulado.

06 Regras de negócio, decisões e processamento em lote

Critério	Decisão implementada
Validade	Foto com URL HTTP(S), latitude/longitude nos intervalos globais válidos, classe reconhecida e confiança entre 0 e 1.
Duplicidade	Mesma classe, distância até 25 m e diferença temporal de até 48 h, comparada a casos aceitos anteriormente.
Confiança	Se inferior a 0,65: revisão humana.
Prioridade	Buraco 85; degrau ou desnível 75; piso tátil 65; obstáculo 55; veículo 50; +5 se houver múltiplas detecções.
Classificação	Alta: 70 ou mais; média: 50 a 69; baixa: abaixo de 50.
IA	Se o assistente recusar, revisão humana. Caso contrário, encaminhamento simulado com fontes retornadas.
Lote	Itera por todos os relatos com status 'recebido'; uma segunda execução ignora os já concluídos.

Observação metodológica: limiares e pontuações foram definidos para demonstrar regras de decisão; não são regras oficiais da Prefeitura nem estimativas comprovadas de risco. A validação da presença física da imagem e a avaliação de sua qualidade pertencem ao MVP/detector.

07 Exceções e recuperação

Falhas temporárias de conexão, HTTP 429 e erros de servidor são reexecutados até três vezes, com espera exponencial. Dados inválidos ou confiança baixa produzem revisão humana, sem interromper o restante do lote. Se o assistente ou o PATCH estiver indisponível após as tentativas, o erro é registrado e o relato fica pendente para uma próxima execução. Os resultados e os estados são consultáveis por API.

08 Logs e auditoria

Cada linha de **logs/auditoria.jsonl** é um JSON com data/hora UTC, identificador da execução, identificador técnico do relato, resultado e motivo, ou código de falha. Ao final, há um resumo de itens, sucessos, falhas, revisões, descartes e encaminhamentos. Não são gravados GPS, URLs de fotos, nome do cidadão ou token de autenticação.

09 Integração com o MVP

O contrato do plano de grupo especifica **GET /relatos**, **POST /assistente** e **PATCH /relatos/{id}/status**. A integração local foi testada com chamadas HTTP a uma API fictícia. O PATCH grava os status e permite que outro componente os consulte. Para os casos encaminhados, arquivos JSON no diretório **outbox/** contêm ID, prioridade, distrito e indicação de canal simulado.

Pendente para a integração final: validar os corpos JSON, credenciais, status, base legal e tratamento da orientação do assistente com os responsáveis pela API. Não foi realizada chamada real a SP156 nem comprovada a conexão à URL pública da equipe.

10 Testes e evidências de validação

Indicador	Resultado observado na simulação local
Itens processados	5
Processamentos bem-sucedidos	5
Encaminhados pelo canal simulado	2
Revisão humana	2
Descartados como possível duplicado	1
Falhas na execução normal	0
Itens na segunda execução	0

A suíte **unittest** contém **13 testes automatizados** cobrindo validações, distância, janela temporal, prioridade, confiança, lote integrado, idempotência, repetição após erro temporário de API, indisponibilidade do assistente, falha em PATCH e ausência de dados sensíveis nos logs. Todos passaram na execução local registrada em **assets/resultado-testes.txt**.

Cenários demonstrados: (a) sucesso com dois encaminhamentos fictícios; (b) decisões com duas revisões e um descarte; (c) falhas simuladas, recuperadas ou registradas sem travar o lote. A API fictícia não substitui teste com o MVP público.

11 Segurança e LGPD

O protótipo usa somente dados sintéticos, não coleta identificadores pessoais nos logs e limita o envio ao assistente ao tipo de barreira e distrito. A chave de API é lida de variável de ambiente e arquivos `.env` são ignorados pelo Git. Antes da produção, o time deve garantir HTTPS, autenticação, autorização, retenção e proteção dos dados no backend. O servidor HTTP de simulação não é apropriado para exposição pública.

12 Publicação, repositório, vídeo e conclusão

GitHub público: [INSERIR LINK] | **Vídeo de até 3 minutos:** [INSERIR LINK] | **URL do MVP:** [INSERIR LINK]

Conclusão: a solução demonstra automação operacional com repetição, gateways de decisão, tratamento de falhas, integração HTTP local e auditoria reproduzível. O trabalho está tecnicamente pronto para a etapa de homologação com a API real, e o vídeo público deverá comprovar a execução no contexto do MVP.